

COMP3050 Team Project Assignment 2026 – API v2

Overview

The project involves building a server which tracks the state of a simple 2D tile-based virtual world. This virtual world could be used for playing a simple adventure game.

Students create and maintain the game world in a server, which communicates with a web-based client using a web API. This document specifies the API version number v2. (The API version number has increased because there are new APIs specified in this document. Also note that the v1 APIs may have also changed in v2 to support new features.)

Note: Students don't create the client or the tile images, which will all be supplied to the students. This is to avoid students getting distracted by the game play elements, because the intention is to focus on the server-side deployment and maintenance.

Game play

An example of game play might look like this:



Map

On the client window, the map view is shown with the player's character in the central location.

The player's view has a fixed size: an 11x11 square grid. But the world map may be larger.

The map has several elements:

- A character which can walk around within the map. A player will see the map with their own character positioned at the central tile.
- There can be multiple players logged in at the same time. (Players do not currently interact with each other, or even see each other. They only share the map.)
- Surfaces, which players can walk on, such as grass, dirt, and various kinds of flooring.
- Impassable elements, such as walls, doors, and water.

Movement

Characters can be moved one space in one of the four cardinal directions:

Direction	Direction name	dy	dx	Description	Client key
N	North	-1	0	One space north.	W or ↑
W	West	0	-1	One space west.	A or ←
S	South	+1	0	One space south.	S or ↓
E	East	0	+1	One space east.	D or →

Diagonal motion and moving more than one space are disallowed.

API

The API supports several operations:

/login	POST: name= <i>NAME</i> encpswr= <i>encrypted_password</i>	Player name and encrypted player game password must be sent in the message body via a POST to the server. The front end client software will encrypt the player game password before sending it to the server. • Returns 200 if the name and password are correct, with a JSON body containing a session token to be used for all further commands. • Returns 400 (Bad Request) if the name and/or password weren't sent. • Returns 401 (Unauthorized) if the name and/or encrypted password sent don't match the server's records.
/logout	?session= <i>SESSION</i>	The server removes its server-side record of the session token, and that token can't be used afterwards. • Returns 200 (OK) after ending the current gaming session for the player identified by the session token. • Returns 401 (Unauthorized) if the session token is invalid or does not identify a currently active player (e.g. if the player has already signed out).
/move	?dy= <i>DY</i> &dx= <i>DX</i> &session= <i>SESSION</i>	Move the character in the corresponding direction if possible. Locations are relative to the character, so dy=0&dx=+1 is East one space, and dy=+1&dx=0 is South one space. • Returns 200 (OK) with confirmation of the new location if successful. • Returns 204 (No Content) if motion is blocked or invalid (e.g. if moving diagonally or more than one space). • Returns 401 (Unauthorized) if the session token is invalid.
/info	?y= <i>Y</i> &x= <i>X</i> &session= <i>SESSION</i>	Fetch information about a particular space, including floor tile kind. The location given in the y and x parameters are the character's absolute location in the map. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction. • Return 200 (OK) with JSON information if successful. • Returns 204 (No Content) if obtaining information about that tile is not allowed (e.g. if the character is not at the location specified). • Returns 401 (Unauthorized) if the session token is invalid.

LOGIN requests

To authenticate a player with the server, a LOGIN request must be sent.

The LOGIN request must be a POST message containing both a player's character name on that server, and an encrypted player game password (see below). If either or both of these are missing, the request is invalid.

The POST message's body is a textual JSON object.

The player's character name is sent in the `name` field. For now, each name is restricted to ASCII letters and hyphens '-', and are case-sensitive. For example, "Baradun", "NPC-Man", and "willow" are valid names, whereas "Greg the Garlic Farmer", "britt95", and "d&elion" are invalid because they contain invalid symbols, spaces, or numbers.

The password corresponding to a player's character is sent in an encrypted form to the server, within the `encpswrd` field. The client-side software encrypts the password. The server does not need to decrypt the password, but instead compares it with an encrypted value on the server.

The encryption scheme is currently very simple, it is merely a human-readable hexadecimal SHA-256 hash of the string formed from the name followed by a semi-colon ';' followed by the password. For example, if the character name was "Baelin" and the password was "Nice day for fishing." the encrypted password would be the SHA-256 hash of "Baelin;Nice day for fishing.", which is "841eb70b9dd5019642751955afbb960a0f741129877db34ceb420a8fb4a9d1dd".

Warning: Because this is not a strong encryption scheme, it is important for testers of the software to never reuse names or passwords from other services, social media, or educational platforms.

If that player's character is not currently logged in, the server will generate a session token and return it to the client in a JSON message. For example:

```
{
  "session": "K3mdn8Ah2j8s099pqDb3Sz32"
}
```

The form that the session token takes is a case-sensitive string of between 20 to 80 ASCII characters, consisting of only uppercase letters, lowercase letters, and numbers. (A hexadecimal number would satisfy this requirement.) The tokens should be sufficiently unpredictable that different players will not be assigned the same session token. A random number may be used to produce a token. Session tokens are opaque to the client; they contain no fields or components that the client can examine.

The server software maintains a list of valid session tokens. When a player logs in, a session token is generated and recorded by the server (e.g. in a key-value store in memory), and sent to the player's client software. When the player logs out, the session token is revoked by the server, so it is removed from the server's records and can no longer be used by that player or any other.

Session tokens are sent with other requests (e.g. MOVE and INFO) and are looked up by the server to validate and find the corresponding player character information. Invalid or missing tokens result in 401 errors for all other API requests.

LOGOUT requests

When a player logs out, the session token is revoked by the server, so it is removed from the server's records and can no longer be used by that player or any other.

If an invalid session token is sent with the LOGOUT request or with any other API request, a 401 error is produced.

However, if the session token is valid, the LOGOUT request causes the server to invalidate and remove that session token from the server's store of session tokens. Any subsequent use of that session token would result in a 401 error.

Ideally, session tokens are never reused, but this is not a requirement.

MOVE requests

Every MOVE request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from MOVE examples below.

The central location where the player's character appears in the view is assigned the number (0,0) in the relative (DY,DX) coordinate system used by MOVE operations. Movement requests for game spaces are relative to character's current location, i.e. relative to that (0,0) view location.

One dy and one dx parameter may be given in each MOVE request. A missing parameter is assumed to be zero. So the following are valid:

Request	Validity	Notes
/move?dy=-1&dx=0	valid	Move N.
/move?dy=-1	valid	Defaults dx=0
/move?dx=+1	valid	Defaults dy=0
/move?dy=0&dx=0	valid	Valid but does nothing. No movement.
/move	valid	Defaults dy=0 and dx=0. No movement.

But the following are invalid:

Request	Validity	Notes
/move?dy=+1&dx=+1	invalid	Diagonal motion is invalid.
/move?dy=-2&dx=0	invalid	Moving more than one space is invalid.

Move requests may be fail for various reasons, such as going beyond the edge of the map, or intersecting a blocking map element such as a brick wall. In such cases, a 204 status code is returned with an empty response body, and no change is made to the server's recorded location of the player's character.

Valid move requests which succeed cause the server's recorded location of the player's character to be updated to the new location, and a 200 status code to be returned, with a JSON response body reporting the new absolute map x and y location of the player's character. For example:

```
{
  "y": 0,
  "x": 6
}
```

INFO requests

Every INFO request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from INFO examples below.

The client uses the INFO request to discover information about locations nearby the character. An INFO request uses absolute (Y,X) locations, relative to the map's co-ordinate system. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction.

If an invalid INFO request is made, the server returns a 204 status code, and the message body will be empty. A request is invalid if the requested x and y values don't match the current player character location.

Upon success, an INFO request returns a 200 status, and the message body will contain JSON data which describes the specified space.

The INFO request's JSON response specifies:

- The "info", a rectangular array (an array of arrays) of floor tile kinds, which the client uses to display the appropriate image tile at that location. Each floor tile kind is a short string.
- The "y" and "x" location that were requested, as integers (which correspond with the character's location).
- The "top", "left", "bottom", and "right" (inclusive) edges locations of the returned rectangular array, as integers.

An example INFO request may be:

```
/info?y=7&x=5
```

An example of an INFO request's JSON response with status 200 might be:

```
{
  "y": 7,
  "x": 5,
  "top": 2,
  "left": 0,
  "bottom": 12,
  "right": 10,
  "info": [
    ["S", "w", "w", "w", "w", "S", "g", "g", "g", "W", "W"],
    ["S", "w", "w", "w", "w", "S", "g", "g", "W", "W", "g"],
    ["S", "S", "S", "w", "S", "S", "g", "g", "W", "g", "g"],
    ["g", "g", " ", " ", "g", "g", "g", "W", "W", "W", "g"],
    [".", "g", "g", " ", "g", "g", "g", "W", "g", "W", "."],
    [" ", " ", " ", " ", "g", "g", "W", "W", "g", "W", "W"],
    [" ", "g", "t", "t", "g", "W", "W", "W", "g", "g", "W"],
    [" ", "g", "g", "t", "g", "W", "W", "g", "g", "g", "W"],
    [" ", "g", "g", "g", "g", "g", "g", "g", "g", "g", "W"],
    [" ", "g", "g", "g", "g", "g", "g", "t", "t", "g", "W", "W"],
    [" ", "g", "g", "g", "g", "g", "g", "t", "g", "g", "W", " "]
  ]
}
```

Some floor kinds block movement. For example, a brick wall prevents a character from moving into that location. Similarly, water prevents character motion.

Floor tile kinds are indicated using short strings. Each letter or symbol in a string specify a tile kind:

Letter or symbol	Tile kind	Blocks movement?
B	brick wall	blocking
b	bridge	-
D	doors closed	blocking
d	doors opened	-
-	dirt	-
f	flagstones	-
g	grass	-
p	pebbles	-
.	one rock	-
,	two rocks	-
:	three rocks	-
;	six rocks	-
S	stone wall	blocking
s	sand	-
t	tree	-
W	water waves	blocking
w	wooden boards	-

The server ensures that MOVE requests fail if that would cause the player's character to intersect a **blocking** map element.

Note that uppercase letters have been used for all **blocking** map elements, but that is a convenience, not a strict rule, and it could change in future versions of this document.

Client-side View

The client displays an 11x11 grid of square locations.

Each location is drawn within the client in the following way:

1. First, the INFO command is used to discover information about that location.
2. Then each floor tile is drawn.
3. Any character at a location is then drawn over the tile.

Vision of a tile is restricted if the location is too far away (more than 5 spaces away in Y or X).

Because a tile is drawn then a character may be drawn over the top, transparent pixels in the character's sprite will reveal the pixels of the tile below.

For speed, when the player moves the character, a check for whether the tile is blocking is performed on the client-side. If the motion would succeed on the client's copy of the map, a MOVE request is made only after this check. This avoids a potentially costly round-trip to the server for every move. However, the client should undo any moves that fail as a result of 204 status codes.

Server-side Map

The server software must maintain a map, and serve rectangular pieces of it to the client as requested (in response to each INFO request).

A simple way to store a map is to use a text file containing the letters/symbols used for floor tile kinds. This text file can be loaded into memory when the server starts, e.g. as a rectangular array of bytes. This avoids the need for a database or anything too complex.

Server-side Sessions

Document Versions

Version 1 of this document was the initial specification, including the MOVE and INFO requests.

This document is at version 2, which introduces the new LOGIN and LOGOUT requests, also adding the requirement for session tokens to be passed to all other APIs.

This document may still be incomplete or lack features. Later versions of this document will supersede earlier versions.